实验四：系统管理与任务调度

- **适用课程**：Linux 操作系统
- **推荐学时**：3 学时 (讲授 1 学时，练习 2 学时)
- **实验平台**：Earendel Web OS [https://earendel.techarts.cn]

实验目标

1. **掌握进程管理核心技能**：熟练使用 `ps`，`top` 监控系统资源占用情况，学会使用 `kill` 与 `killall` 优雅或强制终止异常进程。
2. **理解前后台作业控制**：掌握 `&`，`jobs`，`fg`，`bg` 等命令，理解终端与进程的关联关系，实现多任务并发处理。
3. **熟练配置定时任务**：深入理解 `crontab` 的五星时间调度格式，能够编写并部署周期性自动化运维任务（如自动备份、日志清理）。

Part A：核心知识点解析与自由练习

1. 理论讲解：Linux 进程生命周期与状态

> **Note**
>
> **预备理论**：在 Linux 中，任何运行的程序都是一个进程（Process）。每个进程都有一个唯一的标识符（PID）。系统资源（CPU、内存）都是按进程进行分配调度的。

(1) 进程状态观察（ `ps` 与 `top` ）

- `ps` （Process Status）：查看当前时刻的进程快照。
  - `ps aux`：生产环境最常用。`a` 显示所有终端的进程，`u` 显示进程所有者，`x` 显示没有控制终端的进程。
  - `ps -ef`：以全格式显示进程信息，常配合 `grep` 查找特定进程。例如：`ps -ef | grep nginx`
- `top` / `htop`：动态实时显示系统中各个进程的资源占用状况，类似于 Windows 的任务管理器。
  - 在 `top` 界面中：按 `P` 根据 CPU 排序，按 `M` 根据内存排序，按 `q` 退出。

(2) 进程的终止与信号（ kill ）

kill 命令的本质是向进程发送信号（Signal）。

- kill -15 <PID> (SIGTERM)：默认信号，**优雅终止**。请求进程保存数据并自行退出。
- kill -9 <PID> (SIGKILL)：**强制终止**。内核直接杀死进程，不给进程保存状态的机会。不到万不得已不要使用。
- killall <进程名>： 根据名称杀死所有同名进程。

(3) 前后台作业控制

当我们在终端运行一个耗时极长的命令（如打包几十 GB 的文件），终端会被阻塞。

- 命令 &： 在后台运行该命令。
- Ctrl + Z： 将当前正在前台运行的任务**暂停**并放入后台。
- jobs： 查看当前终端的所有后台任务。
- bg %任务号： 让后台暂停的任务继续在后台运行。
- fg %任务号： 将后台任务调回前台执行。

2. 定时任务调度 (crontab)

系统运维中，有很多任务需要周期性执行，比如每天凌晨 2 点备份数据库。 cron 服务就是为此而生。

(1) crontab 配置文件格式（五星填空法）

每一行代表一个定时任务，格式如下：

```
*  *  *  *  *  command to be executed
T  T  T  T  T
|  |  |  |  |
|  |  |  |  └── 星期 (0 - 7) (0或7代表周日)
|  |  |  └───── 月份 (1 - 12)
|  |  └──────── 日期 (1 - 31)
|  └─────────── 小时 (0 - 23)
└────────────── 分钟 (0 - 59)
```

(2) 常用调度符号

- *： 代表所有可能的值（如在小时位表示“每小时”）。
- ,： 表示多个离散的时间点（如 1,15 * * * * 表示每小时的第1分钟和第15分钟）。
- -： 表示连续的时间范围（如 * 9-17 * * * 表示早上9点到下午5点）。

- `/`：表示频率（如 `*/5 * * * *` 表示每 5 分钟执行一次）。

(3) 经典常用示例

```
# 每分钟执行
* * * * * /path/to/cmd

# 每天凌晨 2:30 执行
30 2 * * * /path/to/cmd

# 每小时的第 0 分钟执行
0 * * * * /path/to/cmd

# 每 5 分钟执行
*/5 * * * * /path/to/cmd

# 每天 9 点到 18 点之间，每 30 分钟执行
*/30 9-18 * * * /path/to/cmd

# 每周一至周五 8:00 执行
0 8 * * 1-5 /path/to/cmd

# 每月 1 号和 15 号凌晨 3 点执行
0 3 1,15 * * /path/to/cmd

# 每周日凌晨 0 点执行
0 0 * * 0 /path/to/cmd
```

(4) `crontab` 管理命令

- `crontab -e`：编辑当前用户的定时任务列表（会自动调用 vim）。
- `crontab -l`：查看当前用户的定时任务列表。
- `crontab -r`：清空当前用户的所有定时任务。

Part B：任务挑战 —— 异常进程排查与自动化备份策略

1. 业务背景

“阿波罗研发中心”的测试服务器近期出现了异常卡顿。经初步排查，实习生前天运行了一个死循环的测试脚本忘记关闭，占用了大量 CPU。
此外，运维总监要求你立刻为上一周配置好的 `/etc/apollo_service.conf` 设定每天的自动备份策略，以

防被意外篡改。
如果这个配置文件找不到了，请新建一个同名的文件，随便写点内容进去即可。

2. 挑战操作步骤

步骤一：模拟异常卡顿进程（后台运行）

在终端中执行以下命令，人为制造一个在后台死循环的 CPU 消耗进程（不要害怕，它只会占用单核）：

```
bash -c 'while true; do let a=1+1; done' &
```

注：命令末尾的 `&` 表示将其直接放入后台运行。终端会返回一个类似 `[1] 12345` 的提示，其中 `12345` 就是它的 PID。

步骤二：定位并猎杀异常进程（`top`，`ps`，`kill`）

1. **监控资源**：输入 `top` 命令，观察 CPU 占用率最高的进程。你会看到一个 `bash` 进程占用了极高的 CPU。记下它的 `PID`（第一列的值）。按 `q` 退出 `top`。
2. **精准查找**：如果你知道脚本名或大概的命令，也可以用 `ps aux | grep while` 来定位它的 PID。
3. **优雅击杀**：使用 `kill` 命令终止该进程。假设查到的 PID 是 12345：

   ```
   kill -15 12345
   ```

4. **验证击杀**：再次运行 `jobs` 或 `ps aux | grep 12345`，确认该死循环进程已经消失。系统恢复流畅。

步骤三：编写备份脚本并测试

1. 在用户家目录创建一个简单的备份脚本 `backup_apollo.sh`：

   ```
   vim ~/backup_apollo.sh
   ```

2. 填入以下内容（包含时间戳生成）：

```
#!/bin/bash
# 获取当前时间，格式为 YYYYMMDD_HHMM
TIMESTAMP=$(date +"%Y%m%d_%H%M")
BACKUP_DIR="/data/apollo_shared/backups"

# 确保备份目录存在
mkdir -p $BACKUP_DIR

# 执行备份
cp /etc/apollo_service.conf $BACKUP_DIR/apollo_service_$TIMESTAMP.conf
```

3. 为脚本赋予可执行权限：

```
chmod +x ~/backup_apollo.sh
```

```
cd ~
./backup_apollo.sh
ls -l /data/apollo_shared/backups/
```

步骤四：配置 Crontab 周期性定时任务

为了在实验课上能立刻观察到效果，我们需要让上述脚本**每隔 3 分钟**自动执行一次。

1. 打开定时任务编辑器：

```
crontab -e
```

(如果系统提示选择编辑器，请按数字选择 `vim.basic`)

2. 在文件末尾添加以下调度规则（注意路径要写绝对路径）：

```
*/3 * * * * /home/hello/backup_apollo.sh
```

(注：系统默认用户为 `hello` 。如果你的用户名不同，请替换为 `pwd` 查看出的真实家目录路径)

3. 保存并退出 (`:wq`)。系统会提示 `crontab: installing new crontab` 。

步骤五：生成调度审计报告 (`sys_audit.txt`)

1. 收集调度与脚本信息，交付给运维总监：

```
# 1. 导出备份脚本内容
echo "--- Backup Script ---" > ~/sys_audit.txt
cat ~/backup_apollo.sh >> ~/sys_audit.txt

# 2. 导出定时任务列表
echo -e "\n--- Crontab List ---" >> ~/sys_audit.txt
crontab -l >> ~/sys_audit.txt

# 3. 导出手动执行的备份结果
echo -e "\n--- Backup Output Verify ---" >> ~/sys_audit.txt
ls -l /data/apollo_shared/backups/ >> ~/sys_audit.txt
```

2. 追加签名：

```
echo -e "\n--- Student Signature ---" >> ~/sys_audit.txt
echo "Student_ID: 20260001" >> ~/sys_audit.txt
echo "Student_Name: ZhangSan" >> ~/sys_audit.txt
```

实验交付与评分标准

交付物

- 最终生成的调度审计报告文件： `~/sys_audit.txt`

评分标准（满分 100 分）

评分维度	分值	考察知识点	评分指标与判定标准
进程异常排查与击杀	30分	`ps` , `top` , `kill`	教师检查系统负载， 死循环进程已不存在且无僵尸进程残留
自动化备份脚本编写	30分	脚本基础, `chmod +x`	脚本逻辑正确，具备可执行权限， 能成功生成带时间戳的备份文件
Crontab 调度策略	25分	`crontab -e` , 时间格式	时间格式 `30 2 * * *` 准确无误， 绝对路径调用正确
交付报告与规范	15分	重定向, 文件整理	`sys_audit.txt` 内容完整，脚本、 任务列表和备份结果呈现清晰，带签名

思考与进阶拓展

1. **权限思考**：如果你的备份脚本里需要备份 `/etc/shadow`（只有 root 能读），普通用户的 crontab 能够成功执行吗？如果不能，应该怎么解决？
2. **任务排错**：很多人写了 crontab 但发现脚本没有被执行，最常见的原因是“环境变量缺失”。查阅资料，了解在 crontab 脚本中为什么建议所有命令（如 `cat`，`cp`，`date`）都写绝对路径？